Experiment - 5

Software Requirements Specification (SRS)

for

Exam Registration System

Table of Contents

1 Introduction

1.1 Purpose
The Exam Registration System is developed to automate and streamline the process of registering for exams in an educational institution. The system will allow students to register for exams efficiently, ensure administrators can manage registrations and schedules, and provide automated notifications. This will reduce paperwork, minimize manual errors, and enhance overall efficiency.

1.2 Document Conventions
This document follows IEEE SRS standards. Functional and non-functional requirements are clearly categorized. The system specifications use standard terminologies relevant to software engineering

1.3 Intended Audience and Reading Suggestions
This Document is intended for:
- **Developers**: Responsible for coding and implementing the system.
- **University Administration**: Managing the exam registration Process.
- **IT Support Team**: Maintaining and Troubleshooting the system.
- **QA/Testers**: Ensuring the system meets requirements.
- **End User**: Students who will register for exams.

1.4 Product Scope
The Exam Registration System is a web-bases application that integrates with the university's student database.
It allows students to-
- Register for exams based on their enrolled courses.
- Pay exam fees (if applicable).
- View their exam schedule.
- Receive notifications and reminders

Administrators can:
- Manage exam registrations.
- Approve or reject applications.
- Generate reports on exam registrations and trends.

1.5 References
Document refer to the following assignment submitted-
Experiment 1
Experiment 3
Experiment 4

2 Overall Description

2.1 Product Perspective
The Exam Registration System is a self-contained, web-based application designed to replace the existing manual process of exam registration within educational institutions. This system will operate as a centralized platform where students, administrative staff, and faculty can interact efficiently for all tasks related to exam enrolment.

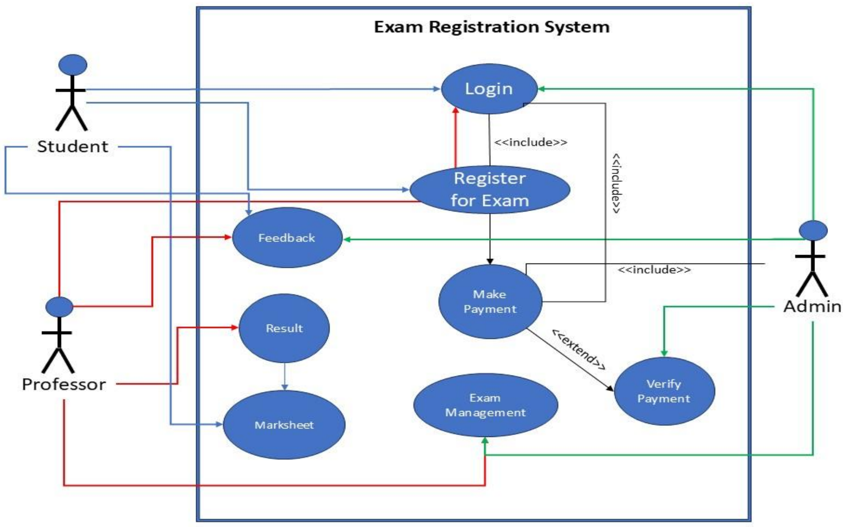
This system is intended to integrate seamlessly with the institution's existing student information management systems, allowing for real-time synchronization of student data such as enrolment numbers, course details, and fee status. It acts as a standalone module but also has the capability to exchange information with other systems like student portals, finance modules (for fee transactions), and notification systems (for SMS/email alerts).

2.2 Product Functions

The system will have the following features:

• **User Authentication**: Secure login for students and administrators.
• **Exam Registration**: Students can register for exams within specified deadlines.
• **Payment Integration** (if required): Secure payment gateway for exam fees.
• **Schedule Management**: Students can check their exam schedules, and administrators can modify them.
• **Notification System**: Automated email/SMS alerts for registration confirmations, deadlines, and schedule updates.
• **Report Generation**: Administrators can generate reports on student registrations and exam trends

2.3 User Class and Characteristics



The following are the actors who perform the use case as stated below-

• **Students**: Basic computer literacy, ability to navigate the system.
• **Administrators**: Knowledge of academic policies and system management.
• **Professor**: Helping the administration to meet the exam results and exam management.

Use Case Name	Description	Pre-Condition	Post-Condition
Login	**User (admin, Student, Faculty, Security personnel)**	**User having account**	**Login Successful**
Register for Exam	**Students Register for their selected exams**	**Successful Login**	**Make Payment**
Make Payment	**Students have to pay Exam Fees for the exam.**	**Successful Registration**	**Payment Verification**

Payment Verifcation	**Check whether the payment made by student is valid or not.**	**Login**	**Successfully Payment Received Message**
Feedback	**Opinion of the student and teacher about the environment of exam**	**After conduction of the Exam**	**Feedback Received**
Exam Management	**The management of exam, Question paper, Invigilator, Assigned rooms, etc.**	**Examined only by Admin**	**Successfully Conduction of Examination**
Result	**The final Marks of the test takers**	**Appeared in exam**	**Download Marksheet**

2.4 Operating Environment

The Exam Registration System is designed to function efficiently in a web-based environment with support for both desktop and mobile platforms. It operates using a client-server architecture, where the client-side interacts through a web browser and the server-side manages business logic, data processing, and storage.

1. Hardware Requirements

The Exam Registration System requires minimal hardware resources on the client side, such as a desktop, laptop, tablet, or smartphone with a 2 GHz dual-core processor and at least 2 GB of RAM. For a smoother experience, especially during peak load times, a 2.5 GHz quad-core processor with 4 GB RAM or more is recommended.

2. Software Requirements

On the client side, the system supports all modern web browsers, including the latest versions of Google Chrome, Mozilla Firefox, Microsoft Edge, and Safari. It is compatible with major operating systems such as Windows, Linux, macOS, Android, and iOS. The server side can run on Linux distributions like Ubuntu or CentOS, or on Windows Server.

3. Network Requirements

The system requires a stable internet connection for both clients and servers. Clients can access the system via broadband or 4G/5G mobile data. For the server, a high-speed internet connection with a static IP is recommended. Alternatively, the application can be hosted on cloud infrastructure such as AWS, Microsoft Azure, or Google Cloud, which ensures scalability, security, and reduced maintenance overhead.

4. Third-Party Integrations

To extend functionality, the system can integrate with third-party services such as payment gateways (e.g., Razorpay, PayPal, or Stripe) for handling secure online transactions.

5. Development and Deployment Environment
The development process will utilize tools like Visual Studio Code for coding, Git for version control, and Postman for API testing. Developers will use browser developer tools for debugging and interface testing.

2.5 Design and Implementation Constraints

1. Programming Language and Framework Constraints
The system must be developed using open-source and widely supported programming languages such as JavaScript (Node.js for backend), HTML, CSS, and modern frontend frameworks like React or Angular.

2. Database Constraints
The system must use a relational or document-based database management system such as MySQL, PostgreSQL, or MongoDB.

3. Platform Dependency
The application must be platform-independent at the user level, functioning seamlessly across different operating systems including Windows, macOS, Linux, Android, and iOS via modern web browsers

4. Security Constraints
Security is a critical constraint in the system design. All user data, including personal information and payment details, must be transmitted over secure HTTPS channels using SSL encryption.

2.6 User Documentation

The following documents shall be prepared-
1. Installation Guide
2. User Manual for end Users

2.7 Assumptions and Dependencies

• Users will access the system through a web browser.
• The university's database is maintained and up to date.
• Third-party services (email, SMS) are available for notifications.

3. External Interface Requirements

3.1 User Interface

The user interface of the Exam Registration System will be web-based and designed with simplicity and accessibility in mind. It will provide an intuitive and responsive layout that works across desktops, tablets, and smartphones. Students will have access to features such as login, registration forms, payment options, and a dashboard to track their application status. Admin users will have access to control panels for managing registration data, generating reports, updating exam schedules, and handling student queries. The design will follow UI/UX best practices, including clear navigation menus, form validations, meaningful error messages, and consistent visual styles using a frontend framework like Bootstrap or Material UI.

3.2 Hardware Interface

The system is accessible through standard input and output devices. On the client side, users can interact with the system using hardware such as a keyboard, mouse, touchscreen, and display monitor. On the server side, the hardware must support multi-user access and be capable of handling concurrent requests during peak registration periods. A web server with at least an 8-core processor, 16 GB RAM, and SSD storage is recommended.

3.3 Software Interface

The system will be compatible with major web browsers including Chrome, Firefox, Safari, and Microsoft Edge. The backend software components will interact with a database management system such as MySQL, PostgreSQL, or MongoDB. APIs will be provided to connect the system with external services like payment gateways, email/SMS notification services, and existing student information systems.

3.4 Communication Interface

Communication between the client and server will occur over HTTP/HTTPS protocols. All sensitive transactions, including login credentials and payment details, will be secured through SSL encryption to ensure data confidentiality and protection against unauthorized access. The system will also be integrated with third-party communication services such as Twilio or SendGrid to send automated emails and SMS notifications regarding registration status, deadlines, and payment confirmations.

4. System Features

4.1 System features and Priority

Feature	Priority
Create an Account	High
Login	High
Apply for Exam	Medium
Feedback	Low
Download Admit Card	High
Download Marksheet	Low
Payment	High
Change Password	Low
View Application	Low

4.2 Functional Requirements

The following is the functional requirements of the system-

1. User Registration and Authentication

The system must allow new users (students and admins) to create secure accounts using valid credentials. Once registered, users must be able to log in through a secure authentication mechanism using their unique username and password.

2. Profile Management

Students should be able to view and update their personal details such as contact information, address, academic program, and semester.

3. Exam Registration
The system must enable students to register for upcoming exams by selecting their subjects or papers from a list relevant to their course and semester. Each registration entry must be validated to ensure eligibility.

4. Fee Payment Integration
The system should support online fee payment for exam registration via secure payment gateways.

5. Real-Time Notifications and Alerts
Students must receive real-time notifications regarding important updates such as registration deadlines, payment confirmation, and changes in the exam schedule.

5. Other Non-functional Requirements

5.1 Performance Requirement
The system must be capable of handling high volumes of simultaneous user requests, especially during peak registration periods. It should support at least 500 concurrent student logins and allow the smooth processing of form submissions and payment operations without significant delay. Page load time should not exceed 2 seconds under normal load, and the database queries must be optimized to return results quickly

5.2 Security Requirement
Security is a critical aspect of the system due to the sensitivity of personal and financial information. All communication between client and server must be encrypted using HTTPS. User authentication will be enforced using a secure login system with password hashing and session management. Role-based access control (RBAC) must be implemented to ensure that only authorized users can access specific functionalities.

5.3 Software Quality Attributes|
The system must be highly reliable, ensuring consistent performance and minimal downtime. Availability should be at least 99.5%, particularly during the critical registration windows. The design must promote usability by providing a clean, intuitive interface that requires minimal training. Maintainability is also essential, so the codebase should follow clean architecture principles with proper documentation to facilitate easy updates and debugging